DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 02
HTTP, REST e modelagem do domínio

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Na Semana 01 vimos que uma API é um contrato. Nesta semana o contrato ganha forma em duas partes. A primeira é o que trafega pela rede: URLs, mensagens HTTP, métodos, status codes e headers, organizados segundo o estilo REST. A segunda é o que o negócio permite: quem pode fazer o quê, por quais estados uma ordem de serviço passa e o que nunca pode ser verdade. As duas partes são desenhadas antes de qualquer linha de código, e é exatamente isso que você vai precisar dizer à IA, porque o CRUD ela gera sozinha.

Conteúdo

1. Objetivos da semana
2. URI e URL: o endereço do recurso
3. A mensagem HTTP
4. Métodos e sua semântica
5. Status codes
6. Headers e negociação de conteúdo
7. Sem estado: sessão e token
8. Recursos, coleções e sub-recursos
9. O modelo de maturidade de Richardson
10. Modelagem do domínio
11. A máquina de estados da ordem
12. Aplicação no ServiceHub
13. Construindo com IA
14. Glossário
15. Exercícios
16. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. decompor uma URL em suas partes e ler uma mensagem HTTP de requisição e de resposta;
2. escolher o método HTTP de cada operação, sabendo quais são seguros, idempotentes e cacheáveis;
3. responder com o status code certo, em especial distinguindo 400, 401, 403, 404 e 409;
4. projetar recursos, coleções e sub-recursos no nível 2 do modelo de Richardson;
5. modelar o domínio do ServiceHub: subdomínios, linguagem ubíqua, entidades, objetos de valor, invariantes e a máquina de estados da ordem.

2. URI e URL: o endereço do recurso

↑ conteúdo

Em REST, cada recurso tem um identificador. O nome técnico desse identificador é **URI** (*Uniform Resource Identifier*, identificador uniforme de recurso), definido na RFC 3986. Existem dois tipos de URI que você vai encontrar:

- **URL** (*Uniform Resource Locator*): uma URI que diz *onde* buscar o recurso, porque traz esquema e host. Exemplo: `https://api.livraria.com/v1/livros/42`.
- **URN** (*Uniform Resource Name*): uma URI que apenas *nomeia*, sem dizer onde buscar. Exemplo: `urn:isbn:9788550819686`. O ServiceHub usa URNs para identificar tipos de erro, como `urn:servicehub:problem:invalid-transition`.

Anatomia de uma URL

```
https://api.livraria.com:443/v1/livros/42?formato=resumo&idioma=pt#capitulo-3
└──┘   └──────────┬─────────┘└┘└─────┬─────┘└──────────┬──────────┘└──────┬──────┘
esquema       host     porta  caminho         query string        fragmento
```

- **Esquema** (`https`): o protocolo; `https` é HTTP cifrado com TLS.
- **Host e porta** (`api.livraria.com:443`): qual máquina e qual porta. Sem porta, vale o padrão do esquema: 80 para `http`, 443 para `https`.
- **Caminho** (`/v1/livros/42`): identifica o recurso dentro do servidor.
- **Query string** (`?formato=resumo&idioma=pt`): pares `chave=valor` separados por `&`, que filtram, ordenam ou paginam.
- **Fragmento** (`#capitulo-3`): fica no cliente, que o usa para rolar a página. Nunca chega ao servidor.

PARA LEMBRAR

Caminho identifica *qual* recurso; query string diz *como* você quer vê-lo. Por isso o número da página de uma listagem vai na query (`/services?page=2`) e o id de um serviço vai no caminho (`/services/12`).

3. A mensagem HTTP

↑ conteúdo

O **HTTP** (*Hypertext Transfer Protocol*) é um protocolo de pedido e resposta: o cliente envia uma **requisição**, o servidor devolve uma **resposta**. A semântica do protocolo, isto é, o significado de métodos, status codes e headers, está na RFC 9110. Toda mensagem, nas duas direções, tem quatro partes: (1) a **linha inicial**, que na requisição traz método, alvo e versão (`GET /v1/livros/42 HTTP/1.1`) e na resposta traz versão, código e frase (`HTTP/1.1 200 OK`); (2) os **headers**, pares `Nome: valor` com metadados; (3) uma **linha em branco**; (4) o **corpo**, opcional, no formato indicado por `Content-Type`. Essa é a forma textual do HTTP/1.1; o HTTP/2 e o HTTP/3 mudam o transporte, mas não a semântica. Um exemplo genérico:

```
GET /v1/livros/42 HTTP/1.1                HTTP/1.1 200 OK
Host: api.livraria.com                    Content-Type: application/json
Accept: application/json                  Content-Length: 38

(sem corpo)                               { "id": 42, "titulo": "Dom Casmurro" }
```

Uma requisição real do ServiceHub

Abrir uma ordem de serviço é um `POST` na coleção de ordens. Note o que o corpo *não* traz: nem o preço, nem o id do cliente. O preço vem do serviço contratado e o cliente vem do token. Voltaremos a isso nas invariantes (seção 10).

Requisição

```
POST /api/v1/service-requests HTTP/1.1
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOi...

{ "serviceId": 12,
  "notes": "Portão azul" }
```

Resposta

```
HTTP/1.1 201 Created
Location: /api/v1/service-requests/31
Content-Type: application/json

{ "id": 31,
  "status": "PENDING",
  "totalPrice": 150.00, ... }
```

Método e URI dizem *o que* se pede; o status code diz *como terminou*; os headers dizem o formato e dão informações extras, como o endereço do recurso criado.

4. Métodos e sua semântica

↑ conteúdo

O método diz a **intenção** do pedido. A RFC 9110 define três propriedades que ajudam a escolher o método certo e que clientes, proxies e caches usam para decidir o que fazer:

- **Seguro:** o cliente não pede nenhuma mudança de estado no servidor; um robô de busca pode chamá-lo à vontade. GET, HEAD e OPTIONS.
- **Idempotente:** repetir o mesmo pedido deixa o servidor no mesmo estado que fazê-lo uma vez. Os seguros, mais PUT e DELETE.
- **Cacheável:** a resposta pode ser guardada e reaproveitada por um tempo, sem perguntar de novo. Em geral, GET e HEAD.

Exemplos do cotidiano: abrir a página de um produto é seguro; "defina o endereço de entrega como X" é idempotente, porque fazer duas vezes dá o mesmo resultado; a foto de capa de um livro é cacheável. Já "some R$ 10 ao saldo" não é idempotente: repetir soma de novo.

> **CONFUSÃO COMUM**
> Idempotência fala do **estado do servidor**, não da resposta. O primeiro `DELETE /services/12` responde 204; o segundo pode responder 404. Mesmo assim o método é idempotente, porque depois de um ou de dois pedidos o serviço está igualmente removido.

Os cinco métodos que a API usa

Método	Intenção	No ServiceHub	Seguro	Idempotente	Sucesso
GET	Ler	Listar ou buscar serviços	sim	sim	200
POST	Criar dentro de uma coleção	Abrir uma ordem de serviço	não	não	201
PUT	Substituir o recurso inteiro	Atualizar os dados de um serviço	não	sim	200
PATCH	Alterar parte do recurso	Mudar só o status da ordem	não	não garante	200
DELETE	Remover	Desativar um serviço	não	sim	204

HEAD é um GET sem corpo; OPTIONS pergunta quais métodos são aceitos e aparece no *preflight* de CORS (Semana 08). O DELETE do ServiceHub é uma **exclusão lógica**: o serviço fica com `active = false`, mas, para quem usa a API, continua sendo um DELETE que responde 204.

Imagine que a conexão caiu e o cliente não sabe se o servidor recebeu o pedido. Repetir `PUT /services/12` com o mesmo corpo deixa o serviço igual. Repetir `POST /service-requests` cria duas ordens. Repetir o `PATCH /service-requests/31/status` de PENDING para ACCEPTED faz a segunda tentativa receber 409, porque a ordem já está em ACCEPTED: é a máquina de estados protegendo

o domínio (seção 11). Conclusão: GET, PUT e DELETE podem ser repetidos sem medo; POST e PATCH exigem cuidado do cliente.

5. Status codes

↑ conteúdo

O status code é um número de três dígitos na primeira linha da resposta. O **primeiro dígito** define a família e diz quem precisa agir. Um cliente que não conhece o código exato deve tratá-lo pela família: um 418 desconhecido ainda é um erro do cliente.

Família	Significado	Exemplos
1xx	Informativo	Recebido, continue. 101 ao trocar de protocolo, como ao abrir um WebSocket.
2xx	Sucesso	200 OK, 201 Created, 204 No Content.
3xx	Redirecionamento	Procure em outro lugar ou use o cache: 301, 304.
4xx	Erro do cliente	O pedido precisa mudar: 400, 401, 403, 404, 409.
5xx	Erro do servidor	O pedido estava certo; o servidor falhou: 500, 502, 503.

A diferença entre 4xx e 5xx tem consequência prática. Um 4xx diz "corrija e tente de novo"; um 5xx diz "o problema é nosso, tente mais tarde". Por isso uma regra de negócio violada **nunca** pode virar 500: o cliente acharia que vale repetir o mesmo pedido.

Os status codes do ServiceHub

Código	Nome	Quando, no ServiceHub
200	OK	Leitura ou atualização, com corpo na resposta.
201	Created	`POST` criou o recurso; o header `Location` aponta para ele.
204	No Content	`DELETE` deu certo; não há corpo.
400	Bad Request	JSON malformado ou campo inválido; o corpo traz `errors: [{field, message}]`.
401	Unauthorized	Sem token, token inválido ou expirado, usuário desativado. Apesar do nome, significa "não autenticado".
403	Forbidden	Autenticado, mas sem permissão: papel errado ou recurso de outra pessoa.
404	Not Found	O recurso não existe, ou o serviço está inativo e quem pede não é o dono nem ADMIN.
409	Conflict	Pedido bem escrito que o estado atual não permite: e-mail repetido, transição inválida, avaliação duplicada.
500	Internal Server Error	Erro inesperado, ou seja, um bug. Nenhuma regra de negócio termina em 500.

Uma forma de lembrar: **401** pergunta "quem é você?" e se resolve com login; **403** diz "sei quem você é, e não pode", e fazer login de novo não resolve.

400, 422 ou 409? Decida uma vez

Há APIs que usam 422 (*Unprocessable Content*) para erros de validação e 400 só para JSON quebrado. Ambas as escolhas são válidas pelo HTTP. O ServiceHub decidiu assim:

- **400 — forma:** o pedido está mal escrito. JSON quebrado, campo obrigatório faltando, preço negativo, status desconhecido.
- **409 — estado:** o pedido está bem escrito, mas o estado atual dos dados não permite. E-mail já cadastrado, transição de COMPLETED para CANCELLED, segunda avaliação da mesma ordem.
- **422:** não usamos.

O importante não é qual número, e sim que a API inteira decida uma vez e documente. Todo erro do ServiceHub sai no formato **ProblemDetail** (RFC 9457), com `Content-Type: application/problem+json`, tema da Semana 04:

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{ "type": "urn:servicehub:problem:invalid-transition",
  "title": "Transição de status inválida", "status": 409,
  "detail": "Não é possível mudar de COMPLETED para CANCELLED." }
```

6. Headers e negociação de conteúdo

↑ conteúdo

Headers são metadados da mensagem; o nome não diferencia maiúsculas de minúsculas. Quatro deles aparecem toda semana:

Header	Direção	Para que serve
`Content-Type`	as duas	Formato do corpo que está *nesta* mensagem. Um POST sem `application/json` recebe 415.
`Accept`	requisição	Formato que o cliente quer *receber*.
`Authorization`	requisição	`Bearer` seguido do token. Ausente numa rota protegida: 401.
`Location`	resposta	Numa resposta 201, a URI do recurso recém-criado.

O ServiceHub também devolve `X-Request-Id` em toda resposta, um identificador para rastrear a requisição nos logs (Semana 15).

Negociação de conteúdo

O mesmo recurso pode ter várias representações: JSON, XML, PDF, em português ou em inglês. Na **negociação de conteúdo**, o cliente diz o que aceita e o servidor escolhe uma representação e declara qual mandou.

```
GET /v1/livros/42                    →   200 OK
Accept: application/json                 Content-Type: application/json
Accept-Language: pt-BR                   Content-Language: pt-BR
```

Quando a negociação falha, há dois códigos diferentes, e é comum confundi-los:

- **406 Not Acceptable:** o servidor não tem nenhuma representação que o `Accept` do cliente aceite. O problema está no que o cliente quer *receber*.
- **415 Unsupported Media Type:** o corpo enviado está num formato que o servidor não lê. O problema está no que o cliente *mandou*.

O ServiceHub só fala JSON. As respostas de sucesso saem como `application/json` e as de erro como `application/problem+json`.

7. Sem estado: sessão e token

↑ conteúdo

O HTTP não lembra o pedido anterior. Cada requisição chega ao servidor como se fosse a primeira. Essa é a restrição **sem estado** (*stateless*) do REST, vista na Semana 01. Então como um usuário "continua logado"? O cliente reenvia uma prova de identidade em *todo* pedido. Há duas formas comuns de fazer isso:

Critério	Sessão com cookie	Token no header
O que viaja	`Cookie: SESSIONID=a81f...` (só um identificador)	`Authorization: Bearer eyJhbGciOi...`
Onde fica o estado	No servidor, numa tabela de sessões	Dentro do próprio token, assinado pelo servidor
Várias instâncias	Precisam compartilhar as sessões	Qualquer uma confere a assinatura
Ponto de atenção	O envio automático abre espaço para CSRF	Revogar antes de expirar é difícil

CSRF (*Cross-Site Request Forgery*) é quando um site malicioso faz o navegador enviar um pedido a outro site, e o navegador anexa o cookie da vítima sozinho. Com o token no header `Authorization`, o navegador não anexa nada automaticamente.

Nenhuma das duas formas é "errada": sessão com cookie funciona bem para sites renderizados no servidor; para APIs consumidas por aplicativos e outros serviços, o token simplifica escalar. O ServiceHub usa token (JWT, Semana 09) e não guarda sessão.

8. Recursos, coleções e sub-recursos

↑ conteúdo

Um **recurso** é qualquer coisa do domínio que mereça um nome: um livro, um autor, a resenha de um livro, até o status de um pedido. A URI identifica o recurso; o JSON é só uma representação dele naquele momento. As URIs de uma API REST se organizam em três formas:

- **Coleção**, conjunto de recursos do mesmo tipo: `/livros`. GET lista; POST cria um item dentro dela.
- **Item**, um elemento identificado: `/livros/42`. GET lê, PUT substitui, PATCH altera parte, DELETE remove.
- **Sub-recurso**, que só existe no contexto do pai: `/livros/42/resenhas`.

No ServiceHub: `/services` (coleção), `/services/12` (item), `/services/12/reviews` (avaliações daquele serviço) e `/service-requests/31/status` (o status daquela ordem).

Ações que parecem verbos

Aceitar, iniciar, concluir e cancelar uma ordem parecem ações, e a tentação é criar uma rota para cada uma. A saída REST é tratar o **status como sub-recurso**: a ação passa a ser uma alteração parcial do estado da ordem.

Verbos na URI

```
POST /acceptRequest?id=31
POST /service-requests/31/cancel
GET  /startOrder/31
```

Cada ação vira uma rota nova, e o método HTTP perde o significado.

Status como sub-recurso

```
PATCH /service-requests/31/status
Content-Type: application/json

{ "status": "ACCEPTED" }
```

Uma rota só; o destino vai no corpo e a máquina de estados decide se vale.

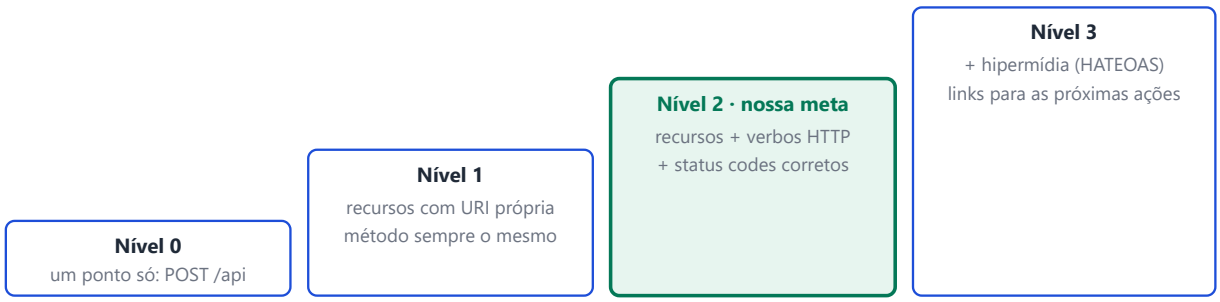
CONFUSÃO COMUM

Aninhar demais, como em `/autores/3/livros/42/resenhas/7`, acopla o cliente à hierarquia: para ler uma resenha ele precisa saber o autor e o livro. Use sub-recurso quando a relação é de posse, e no máximo um nível. Se o item tem id próprio, prefira `/resenhas/7`. É por isso que o ServiceHub tem `GET /reviews/{id}`.

9. O modelo de maturidade de Richardson

↑ conteúdo

Leonard Richardson propôs uma escala para classificar o quanto uma API usa a Web a seu favor, e Martin Fowler a popularizou num artigo. É uma régua didática, não uma norma: cada nível acrescenta uma parte da interface uniforme do REST.



Cada degrau acrescenta uma parte da interface uniforme. O ServiceHub mira o nível 2.

O nível 3, chamado **HATEOAS** (*Hypermedia as the Engine of Application State*), é a parte menos adotada da interface uniforme. Pelo critério de Fielding, só ele seria REST de fato; na prática, a maioria das APIs ditas REST está no nível 2, e é esse o nosso alvo.

10. Modelagem do domínio

↑ conteúdo

Até aqui tratamos da forma: como a API conversa. Agora tratamos do conteúdo: o que o sistema permite. Essa parte a IA não sabe, porque é específica do seu negócio. As ideias desta seção vêm do **DDD** (*Domain-Driven Design*, projeto orientado ao domínio), uma abordagem que coloca o modelo do negócio no centro do software.

Domínio e subdomínios

Domínio é a área de negócio que o software atende. Dentro dele há **subdomínios**, e nem todos merecem o mesmo esforço:

Tipo	O que é	No ServiceHub
Principal (*core*)	O que a empresa faz diferente dos concorrentes. Lógica complexa, feita em casa, com o melhor time.	A contratação e o ciclo de vida da ordem.
De suporte	Necessário ao negócio, mas sem vantagem competitiva. Costuma ser simples, quase um cadastro.	Catálogo de serviços e avaliações.
Genérico	Todo mundo resolve do mesmo jeito. Use solução pronta e testada.	Autenticação com JWT e senha com BCrypt.

Exemplo genérico: num aplicativo de entrega de comida, o principal é casar pedido, restaurante e entregador com rapidez; o cardápio de cada restaurante é de suporte; o pagamento com cartão é genérico, resolvido por um provedor. A consequência prática é que o esforço de modelagem e de teste vai para o subdomínio principal. É ali também que a IA mais erra, porque a regra é específica do negócio; no genérico ela costuma acertar, porque o padrão é público.

Linguagem ubíqua

Linguagem ubíqua é a linguagem do negócio, usada do mesmo jeito por especialistas, desenvolvedores, código, testes e documentação. Sem tradução no meio, não há o que se perder.

Termo do negócio	No código · no banco	Atenção
Serviço	`ServiceOffering` · `services`	O nome evita conflito com a anotação `@Service` do Spring.
Ordem de serviço	`ServiceRequest` · `service_requests`	Não confundir com requisição HTTP.
Cliente da ordem	campo `client` da ordem	Pode ser um PROVIDER contratando outro prestador.

CONFUSÃO COMUM

"Cliente" tem dois sentidos no ServiceHub: o **papel** CLIENT de um usuário e o **cliente de uma ordem**, quem a abriu. Misturar os dois gera o erro de impedir um PROVIDER de contratar. Termo ambíguo vira código ambíguo. Um termo só precisa ter significado único dentro de um **contexto delimitado**, a fronteira em que aquele modelo vale.

Entidade, objeto de valor e invariante

Entidade

Tem **identidade própria**, que dura enquanto os dados mudam. Uma conta bancária continua a mesma quando o saldo muda; duas contas com o mesmo saldo não são a mesma conta. No ServiceHub: usuário, serviço, ordem, avaliação. Em Java, vira uma classe `@Entity` com `id`.

Objeto de valor

Definido **só pelos atributos** e imutável: muda trocando por outro. R$ 150,00 é igual a qualquer outro R$ 150,00; o mesmo vale para um endereço ou um intervalo de datas. No ServiceHub: preço, e-mail, nota de 1 a 5. Em Java, cabe bem num `record`.

Uma **invariante** é uma condição que precisa ser verdadeira antes e depois de qualquer operação. Exemplo genérico: o saldo de uma conta sem limite nunca fica negativo; quem tenta quebrar a regra recebe um erro, e o dado não muda. As invariantes do ServiceHub estão no Mapa de Construção como regras RN01 a RN13. Algumas delas:

Regra	Enunciado	Resposta
RN01	E-mail é único, sem diferenciar maiúsculas.	409
RN06	Ninguém contrata o próprio serviço; o serviço precisa estar ativo.	409
RN07	O preço da ordem é o do serviço na criação e não muda depois.	—

Regra	Enunciado	Resposta
RN09	Só quem abriu a ordem avalia, só depois de COMPLETED, uma única vez.	403 · 409
RN10	Usuário ou serviço com ordens ativas não pode ser excluído.	409

A RN07 explica por que o corpo do `POST /service-requests` não traz preço: aceitar o preço vindo do cliente seria **mass assignment**, deixar o cliente definir um campo que só o servidor deveria definir. Congelar o preço na ordem garante que um reajuste amanhã não muda o que foi contratado hoje.

Quem pode fazer o quê

Cadastrar-se: qualquer pessoa, como CLIENT ou PROVIDER (RN03). Publicar serviço: PROVIDER, sempre em nome de quem está logado (RN04). Alterar ou desativar serviço: o dono ou um ADMIN (RN05). Abrir ordem: CLIENT ou PROVIDER, nunca no próprio serviço; ADMIN não abre ordens (RN06). Mudar status: depende do estado e do **papel na ordem** (RN08), que é diferente do papel do usuário: um PROVIDER pode ser o cliente de uma ordem. Avaliar: só quem abriu a ordem, depois de concluída (RN09).

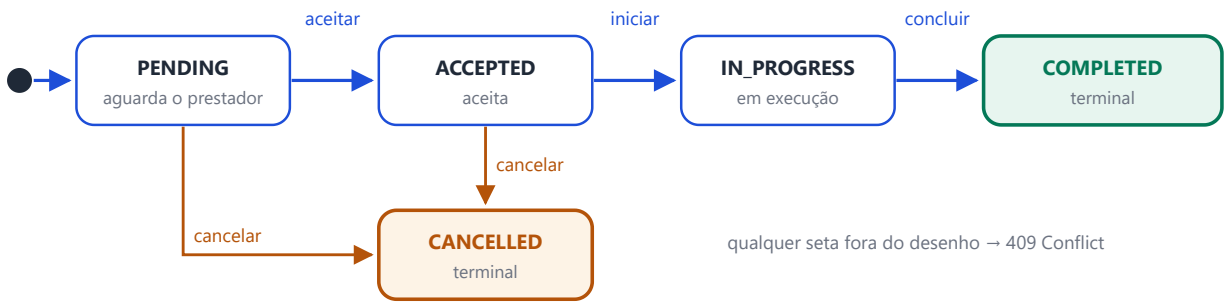
11. A máquina de estados da ordem

↑ conteúdo

Uma **máquina de estados** é um modelo com um conjunto finito de estados, um estado inicial, eventos que levam de um estado a outro e estados **terminais**, de onde não se sai. A regra de ouro: *o que não está desenhado é proibido*. Desenhada (em geral com a notação de estados da UML), a regra fica explícita e testável; no código, vira uma tabela de dados, e não `if`s espalhados.

Exemplo genérico, um pedido de loja virtual: CRIADO → PAGO → ENVIADO → ENTREGUE. CRIADO e PAGO também podem ir para CANCELADO; ENVIADO não, porque a mercadoria já saiu. Com n estados existem $n \times n$ pares possíveis; a máquina diz quais são válidos.

A vida de uma ordem de serviço



Máquina de estados da ordem de serviço do ServiceHub. COMPLETED e CANCELLED são terminais.

A tabela de transições é a regra

De	Para	Quem pode pedir
PENDING	ACCEPTED	Prestador dono do serviço
PENDING	CANCELLED	Cliente da ordem ou prestador
ACCEPTED	IN_PROGRESS	Prestador dono do serviço
ACCEPTED	CANCELLED	Cliente da ordem ou prestador
IN_PROGRESS	COMPLETED	Prestador dono do serviço
Qualquer outra combinação		409 Conflict, com `detail` explicando a transição recusada

409: fora da tabela

O pedido não faz sentido no estado atual, por exemplo COMPLETED → PENDING. Vale até para ADMIN. O cliente da API deve **corrigir o fluxo**.

403: não é o seu papel

A transição existe, mas não para você, por exemplo o cliente tentando aceitar a própria ordem. O cliente da API deve **trocar de usuário**. ADMIN pode qualquer transição válida.

Quem não participa da ordem recebe 403 *antes* de qualquer análise, para não revelar a um estranho em que estado ela está.

O banco tem um `CHECK` que só aceita os cinco valores de status, mas não sabe *quem* pode ir de *qual* estado para *qual*. Essa regra vive na camada de serviço, e cada transição precisa de teste.

12. Aplicação no ServiceHub

↑ conteúdo

Nesta semana a equipe ainda não escreve código. Mesmo assim vale ver como o desenho vira Java no projeto de referência. Os trechos são reais e foram resumidos.

A tabela de transições como dado

O enum `RequestStatus` guarda a máquina como um mapa de pares (de, para) para os atores que podem pedir (resumido):

```
public enum RequestStatus {
    PENDING, ACCEPTED, IN_PROGRESS, COMPLETED, CANCELLED;

    public enum Actor { CLIENT, PROVIDER, ADMIN }
    private record Rule(RequestStatus from, RequestStatus to) { }

    private static final Map<Rule, Set<Actor>> TRANSITIONS = Map.of(
        new Rule(PENDING, ACCEPTED),     EnumSet.of(Actor.PROVIDER),
        new Rule(PENDING, CANCELLED),    EnumSet.of(Actor.CLIENT, Actor.PROVIDER),
        new Rule(ACCEPTED, IN_PROGRESS), EnumSet.of(Actor.PROVIDER),
        new Rule(ACCEPTED, CANCELLED),   EnumSet.of(Actor.CLIENT, Actor.PROVIDER),
        new Rule(IN_PROGRESS, COMPLETED), EnumSet.of(Actor.PROVIDER));

    public boolean canTransitionTo(RequestStatus target) { ... }  // par existe?
    public boolean allowedFor(RequestStatus target, Actor actor) {
        ... // ADMIN pode qualquer par existente; os demais, só os da tabela
    }
}
```

A ordem das verificações

Em `ServiceRequestService.changeStatus`, a ordem é: ordem inexistente → 404; quem chama não participa → 403; `canTransitionTo` falso → `InvalidStateTransitionException` (409); `allowedFor` falso → `AccessDeniedException` (403).

E o controller devolve `ResponseEntity.created(URI.create("/api/v1/service-requests/" + body.id()))`, que gera o 201 com `Location`. O DTO de entrada `ServiceRequestCreateDTO` tem só três campos, `serviceId`, `scheduledAt` e `notes`: não existe campo de preço, de cliente nem de status para o cliente preencher (RN07 e mass assignment). A decisão está registrada no ADR-0006 do projeto; ADR é tema da Semana 03.

Como fica a entrega da equipe

Os dois documentos da semana são Markdown no repositório. No `docs/endpoints.md`, cada linha traz método, rota, quem pode, sucesso e cada erro, como na seção 6 do Mapa; um diagrama em que tudo responde 200 está errado. O diagrama de estados pode ser escrito em **Mermaid**, notação de diagramas em texto que o GitHub desenha ao exibir o arquivo (início do exemplo):

````
```mermaid
stateDiagram-v2
    [*] --> PENDING
    PENDING --> ACCEPTED: prestador aceita
    PENDING --> CANCELLED: cliente ou prestador
    ...as outras três setas da tabela...
```
````

13. Construindo com IA

↑ conteúdo

Nesta semana ainda não há código, e isso é proposital. Peça um CRUD à IA e ela gera um em segundos; peça a regra de quem aceita a ordem e ela inventa uma. A especificação é sua.

Movimento	O que você faz nesta semana
Especificar	A equipe escreve, sem IA, a tabela de transições, a de "quem pode" e as invariantes.
Gerar	A equipe monta `docs/endpoints.md` e `docs/dominio.md`; a IA, no máximo, converte a tabela em Mermaid.
Revisar	A IA revisa os dois documentos contra o Mapa; você confere cada apontamento.
Provar	Sem código, a prova é a tabela de casos esperados dos 5 × 5 pares de estados.

Prompt fraco e prompt bom

Fraco

```
faz os endpoints de uma API de ordens de serviço
```

A IA não conhece as regras do ServiceHub: inventa transições, cria rotas com verbos, responde 200 para tudo e ainda gera código.

Bom

Dá contexto e versões, entrega o Mapa e os rascunhos, pede só revisão, diz o que conferir e o formato da resposta. Texto completo abaixo.

O prompt desta semana

Cole o prompt abaixo junto com as seções 4, 5, 6 e 9 do Mapa de Construção e com os dois rascunhos da equipe.

```
Contexto: somos uma equipe de Engenharia de Software projetando a
API ServiceHub (Java 21, Spring Boot 4.1, PostgreSQL 16). Ainda não
há código: nesta etapa entregamos dois documentos Markdown no
repositório, docs/endpoints.md (diagrama de endpoints) e
docs/dominio.md (máquina de estados da ordem, tabela de transições,
quem pode fazer o quê e invariantes). O contrato oficial é o Mapa de
Construção, cujos trechos colamos abaixo.

Objetivo: revisar os nossos dois documentos contra o Mapa.

Restrições:
- NÃO gere código Java, SQL nem configuração.
- NÃO reescreva os documentos; aponte problemas e sugira a correção
  de cada um em uma linha.
- Use só o Mapa como fonte; o que não estiver lá é "não especificado".
- Regras do projeto: todas as rotas em /api/v1; sem verbos na URI;
  POST de criação responde 201 com Location; DELETE responde 204;
  400 é erro de forma (validação), 409 é erro de estado (regra de
  negócio ou transição inválida), 403 é papel ou dono errado;
  nenhuma regra de negócio responde 500; não usamos 422.
- Máquina de estados: PENDING→ACCEPTED, ACCEPTED→IN_PROGRESS,
  IN_PROGRESS→COMPLETED (prestador); PENDING ou ACCEPTED→CANCELLED
  (cliente da ordem ou prestador); ADMIN pode qualquer transição
  válida; COMPLETED e CANCELLED são terminais; qualquer outra
  combinação é 409; papel errado em transição válida é 403; quem não
  participa da ordem recebe 403 antes da análise.

Antes de revisar, mostre o seu plano (a lista de verificações, na
ordem) e espere eu confirmar.

Depois, responda em duas tabelas (uma por documento) com as colunas:
item | o que está no nosso documento | o que diz o Mapa | correção.
Confira, no mínimo: cada rota da seção 6 do Mapa está presente; cada
rota tem quem pode, sucesso e todos os erros; as setas do diagrama de
estados são exatamente as cinco da tabela; os estados terminais não
têm saída; cada RN01–RN13 aparece em docs/dominio.md.

Critério de aceite: a revisão cobre 100% das rotas do Mapa e as 25
combinações de estado; cada apontamento cita a seção do Mapa; nenhum
trecho de código na resposta.

Trechos do Mapa:
<cole aqui as seções 4, 5, 6 e 9>

Nossos documentos:
<cole aqui docs/endpoints.md e docs/dominio.md>
```

O que a IA costuma errar nesta etapa

- **Status livre:** aceitar qualquer status no PATCH, inclusive o cliente aceitar a própria ordem, ou permitir sair de COMPLETED.

- **500 para tudo:** "não encontrado" vira 500, o erro mais comum em APIs iniciantes e em código gerado por IA.
- **Verbos na URI:** `/acceptRequest`, `/cancelOrder`; a ação deveria ir no corpo do `PATCH /status`.
- **Preço e dono no corpo:** aceitar `totalPrice`, `clientId` ou `providerId` vindos do cliente, em vez de tirá-los do serviço e do token.
- Gerar código quando o pedido era revisão, ou "concordar" sem conferir rota por rota.

Checklist de revisão

☐ Todas as rotas da seção 6 do Mapa estão no `docs/endpoints.md`, com prefixo `/api/v1`?

☐ Todo POST de criação tem 201, todo DELETE tem 204, e nenhuma linha responde só 200?

☐ 409 aparece nas regras de estado e 403 nas de papel ou dono, sem trocar um pelo outro?

☐ O diagrama de estados tem exatamente cinco setas, e COMPLETED e CANCELLED não têm saída?

☐ As treze regras RN01–RN13 estão no `docs/dominio.md`?

Prova

Monte a tabela de casos esperados: para cada um dos 25 pares (de, para) e cada papel na ordem (cliente, prestador, ADMIN, terceiro), a resposta esperada (200, 403 ou 409). Ela está certa quando exatamente cinco pares respondem 200 para o prestador, dois respondem 200 para o cliente, e todo par com origem em COMPLETED ou CANCELLED responde 409 para quem participa da ordem. Essa tabela vira teste automatizado nas próximas semanas.

14. Glossário

↑ conteúdo

URI	Identificador uniforme de um recurso; URL e URN são tipos de URI.
Método seguro	Método que não pede mudança de estado no servidor: GET, HEAD, OPTIONS.
Idempotente	Repetir o pedido deixa o servidor no mesmo estado que fazê-lo uma vez.
Negociação de conteúdo	Escolha da representação a partir do que o cliente aceita (`Accept`).
Stateless	O servidor não guarda sessão; cada pedido traz tudo o que é preciso.
CSRF	Ataque em que um site faz o navegador enviar um pedido com o cookie da vítima.
Sub-recurso	Recurso que só existe no contexto de outro, como `/services/12/reviews` .
HATEOAS	Respostas com links para as próximas ações; nível 3 de Richardson.
Subdomínio principal	Parte do negócio que o diferencia dos concorrentes e merece o maior esforço.
Linguagem ubíqua	Vocabulário do negócio usado igual por pessoas, código, testes e documentos.
Entidade	Objeto com identidade própria, que persiste enquanto os dados mudam.
Objeto de valor	Objeto imutável definido só pelos atributos, como um preço.
Invariante	Condição que precisa ser verdadeira antes e depois de qualquer operação.

15. Exercícios

↑ conteúdo

1. Decomponha a URL `http://localhost:8090/api/v1/services?category=ELETRICA&page=1#topo` em esquema, host, porta, caminho, query string e fragmento. Qual dessas partes o servidor nunca recebe?
2. Classifique cada operação como segura, idempotente (mas não segura) ou nenhuma das duas: (a) "defina o telefone como 62999990000"; (b) "transfira R$ 50 para a conta 7"; (c) "mostre o extrato"; (d) "apague a avaliação 4".
3. O primeiro `DELETE /api/v1/services/12` respondeu 204 e o segundo respondeu 404. Um colega diz que isso prova que o DELETE não é idempotente. Ele está certo? Justifique.
4. Indique o status code para cada caso no ServiceHub: (a) cadastro com e-mail já existente; (b) `POST /services` sem token; (c) um CLIENT tenta `POST /services` ; (d) preço `-10` num serviço; (e) `GET /services/999` inexistente; (f) prestador pede COMPLETED numa ordem PENDING.
5. Um cliente envia um POST com `Content-Type: text/plain` . A API deve responder 406 ou 415? E se o cliente pedir `Accept: application/xml` numa API que só fala JSON?
6. Explique por que, com sessão guardada na memória do servidor, duas instâncias da API atrás de um balanceador podem "deslogar" o usuário. Como o token resolve isso?

7. Classifique a API abaixo no modelo de Richardson e reescreva-a no nível 2: `POST /api` com corpo `{"acao": "cancelarOrdem", "id": 31}`.

8. Classifique como entidade ou objeto de valor: (a) a ordem 31; (b) o valor R$ 150,00; (c) o e-mail `maria@x.com`; (d) o usuário Maria. Justifique uma delas.

9. Para cada pedido de mudança de status, diga a resposta (200, 403 ou 409) e por quê: (a) o cliente da ordem pede ACCEPTED numa ordem PENDING; (b) o prestador pede CANCELLED numa ordem IN_PROGRESS; (c) o ADMIN pede PENDING numa ordem COMPLETED; (d) um usuário que não participa da ordem pede CANCELLED numa ordem PENDING.

10. O corpo de um `POST /service-requests` chegou com `"totalPrice": 1.00`. O que a API deve fazer com esse campo, e qual regra do Mapa justifica?

11. Escreva um prompt pedindo à IA que gere, a partir da tabela de transições, a tabela de casos esperados dos 25 pares de estado para o papel "prestador". Inclua contexto, restrições e critério de aceite, e diga como você conferiria a resposta.

16. Para estudar

↑ conteúdo

Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- RODRIGUES, T. N. et al. *Integração de aplicações*. Grupo A, 2020. Capítulos "Geração de serviços com REST" e "Serviços web".
- KHONONOV, V. *Aprenda Domain-Driven Design*. Alta Books, 2024. Capítulos 1 a 3 (subdomínios, linguagem ubíqua, contextos delimitados). Se der tempo para só uma leitura, que seja o capítulo 2.
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software*. 9. ed. Grupo A, 2021. Capítulos 7 e 8.

Referências abertas:

- FIELDING, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. Tese (Doutorado), University of California, Irvine, 2000. Capítulo 5.
- RFC 9110, *HTTP Semantics*: seções de métodos e de status codes. RFC 3986, *URI: Generic Syntax*.
- RFC 9457, *Problem Details for HTTP APIs* (Semana 04). MDN Web Docs, guia de HTTP: `developer.mozilla.org/pt-BR/docs/Web/HTTP`. Documentação do Mermaid: `mermaid.js.org`.